

COMP4423 - Pet Mischief Project

PATEL Diya (25142309x)

Team Members	Student ID
PATEL Diya	25142309x
RADHAKRISHNAN Aishwarya	25142316x

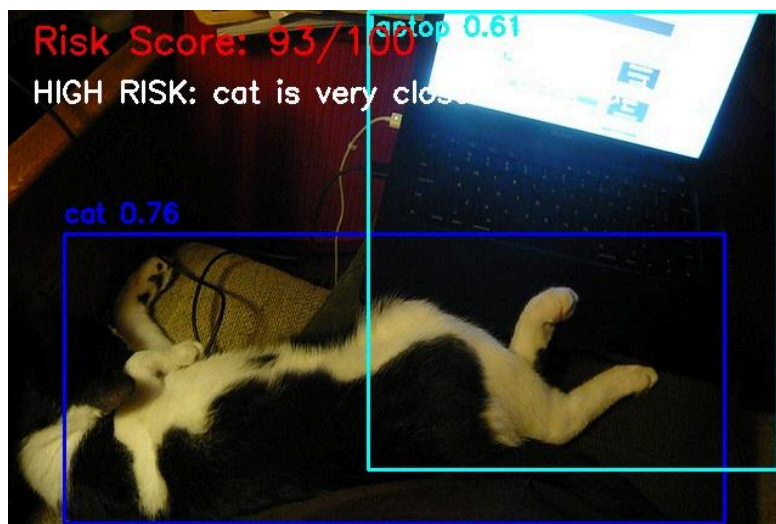
1. Introduction

The goal of this project was to build a Pet Mischief Detector, an end-to-end computer vision system that can detect pets and household objects in an image and then estimate whether the scene represents a potentially mischievous situation. Instead of only recognizing objects independently, the project also required the system to reason about spatial relationships between pets and nearby objects and convert that information into a risk score and warning message. According to the project brief, the system should be able to identify and locate pets and household objects, then use this spatial information to produce context-aware outputs such as low-risk or high-risk warnings.

Our group implemented the project using an object detection approach based on YOLOv8. We first curated a custom dataset by filtering relevant classes from the COCO dataset, then split and exported the dataset into YOLO format for training. After that, a pretrained YOLOv8 model was fine-tuned on the custom dataset. Finally, the model outputs were passed to a custom “mischief logic” module that estimated a risk score based on the proximity between pets and risky household objects.

The final system produces two kinds of outputs for each image: object detections and a scene-level mischief assessment. This makes the project more meaningful than standard object detection alone, because the output depends not only on what is present in the image, but also on how the detected objects are spatially related. In this way, the project combines object detection with simple scene reasoning in a practical and interpretable way.

Figure 1: Example of output image showing detections and risk label, cat is close to laptop, high risk



2. Method

a. System design overview

The entire process was created using four primary steps. First, there was a data pipeline involving curating data sets containing both pets and domesticated items. Second, the data sets were divided into training, testing, and validation and converted to YOLO format. Third, YOLOv8 detection model was used to train the data sets. Fourth, results of this training were sent to the mischief scoring component which created a risk score and warning message.

This architecture has been chosen due to the separation between different elements in the project, which makes testing more feasible. The detector's job is the detection and localization of any object in the scene, whereas the mischief logic is used for scene interpretation. Debugging became much easier using this approach.

b. Class definition and dataset curation

The first step was defining the scope of the detector by selecting classes that were realistic for pet mischief scenarios. The final class set included both pet classes and household object classes: cat, dog, cup, laptop, potted plant, book, tv, remote, frisbee, and vase.

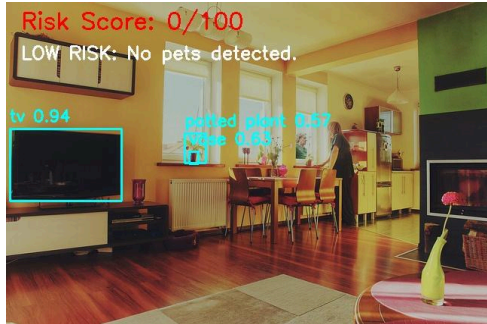
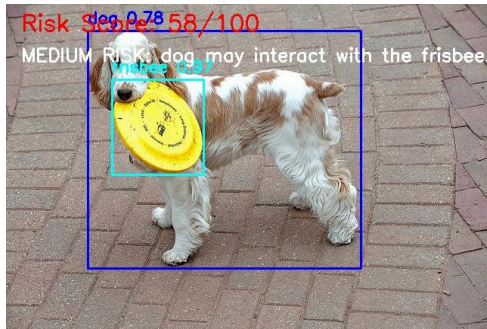
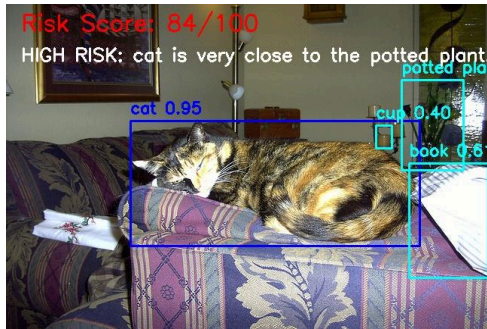
The above classes were picked because they are frequently observed in a house setting and have an ability to cause mischief. For instance, the combination of pets near cups can be indicative of possible spills, while pets near computers will indicate the possibility of damage. Similarly, pets around flower vases will imply the likelihood of tipping over.

To compile the dataset, we used the COCO 2017 data via FiftyOne and kept the desired classes only. This approach allowed us to obtain a realistic dataset with existing annotations while avoiding the time cost of manually collecting and labeling all images from scratch. The dataset includes diverse indoor and outdoor scenes, different viewpoints, cluttered backgrounds, and occlusion, which made it suitable for testing the robustness of the system in more realistic conditions.

Table 1. Final selected classes for the Pet Mischief Detector.

Category Type	Classes
Pet	Cat, dog
Objects	Cup, laptop, potted plant, book, tv, remote, frisbee, vase

Figure 2. Example images from the curated dataset.

Low risk - no pets detected	 Risk Score: 0/100 LOW RISK: No pets detected. tv 0.94 potted plant 0.87 table 0.91
Low risk - pet detected, but no risky object nearby	 Risk Score: 10/100 LOW RISK: Pet detected, but no risky object nearby. dog 0.90
Medium risk	 Risk Score: 58/100 MEDIUM RISK: dog may interact with the frisbee. dog 0.78 frisbee 0.95
High risk	 Risk Score: 84/100 HIGH RISK: cat is very close to the potted plant. cat 0.95 potted plant 0.87 cup 0.40 book 0.81

c. Dataset splitting and formatting

After the dataset was curated, it was split programmatically into training, validation, and test sets using an 80/10/10 ratio. A fixed random seed was used so that the split would be reproducible. This was important because the project specifically required the split to be programmatic and reproducible for fair evaluation.

After splitting, the dataset was exported into YOLO format. This created separate image and label folders for the three splits, together with a dataset.yaml file describing the classes and paths. The final structure followed the format expected by the Ultralytics YOLOv8 training pipeline.

This stage was important because it ensured the dataset was properly organized for both training and evaluation. It also reduced the chance of accidental data leakage between train and test sets.

Table 2. Dataset split summary.

Total number of images	1200
Training images	960
Validation images	120
Test images	120

Figure 3. Exported YOLO dataset structure showing the dataset.yaml file and the train, validation, and test image splits

loads > pet_mischief_data > pet_mischief_data

Name	Date modified	Type	Size
 images	4/20/2026 9:29 AM	File folder	
 labels	4/20/2026 9:29 AM	File folder	
 dataset	4/20/2026 9:29 AM	YAML File	1 KB

ids > pet_mischief_data > pet_mischief_data > images

Name	Date modified	Type	Size
test	4/20/2026 9:29 AM	File folder	
train	4/20/2026 9:29 AM	File folder	
val	4/20/2026 9:16 AM	File folder	

loads > pet_mischief_data > pet_mischief_data > labels

Name	Date modified	Type
test	4/20/2026 9:29 AM	File folder
train	4/20/2026 9:29 AM	File folder
val	4/20/2026 9:16 AM	File folder

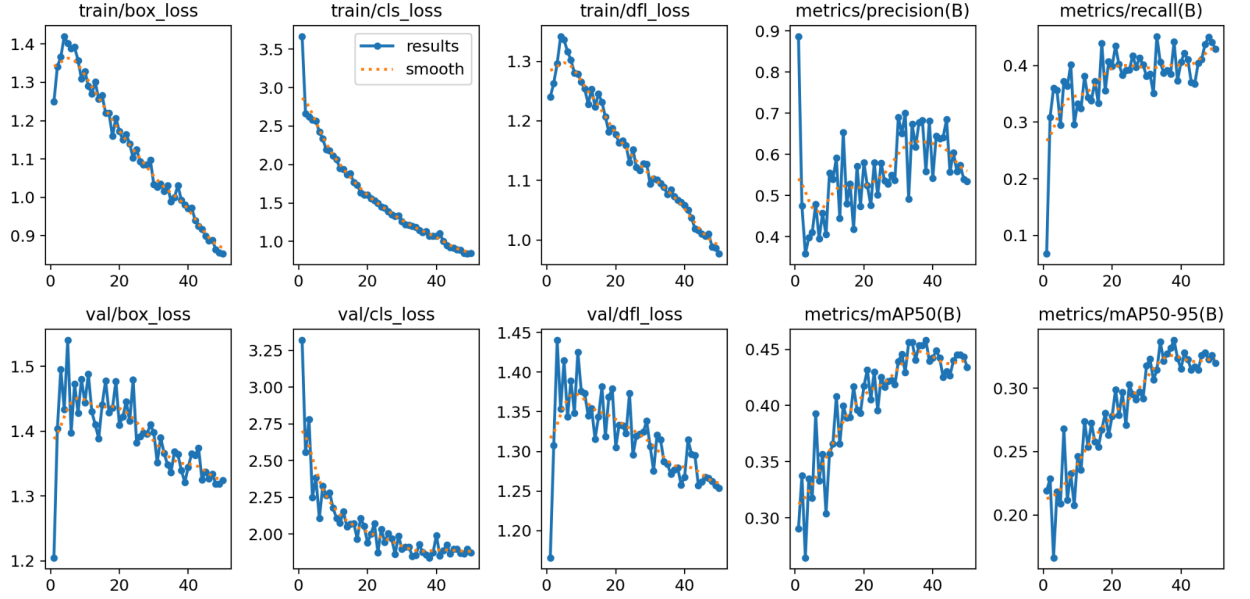
d. Model Training

The object detector used in this project was YOLOv8, initialized from the pretrained yolov8n weights. This model was chosen because it is lightweight, efficient, and well suited for building an end-to-end detection pipeline in Colab. Since our custom dataset was much smaller than the full COCO dataset, transfer learning was an appropriate choice because it allowed the model to begin with useful general visual features and then adapt them to our selected pet and household-object classes.

Training for the model was performed using the customized dataset that was exported to YOLO format. The training process involved setting the input image size to 640 pixels, batch size to 16, and epochs to 50. This set up gave a good trade-off between training time and detection efficiency. Using the Ultralytics YOLOv8 framework also made the training process easy to manage, since it automatically saved the training logs, metric plots, and best-performing model weights.

Although the main training process was handled by my teammate, I contributed by helping debug parts of the training pipeline and by refining the final class set so that the detector would better support the later mischief-scoring logic.

Figure 4: YOLOv8 training curves for the Pet Mischief Detector.



e. Mischief logic design

After object detection, the second stage of the system estimated whether the scene was risky. The detector output gives class labels, confidence scores, and bounding boxes. Our custom mischief logic used this information to identify pets and nearby household objects, then calculate a risk score based on object type, relative distance, and box overlap.

Different objects were assigned different risk weights. More fragile or costly objects, such as laptops and vases, were treated as higher-risk than books or frisbees. The center distance between the pet box and the object box was normalized by the image diagonal so that the scoring would be more consistent across images of different sizes. If the pet and object boxes overlapped, an additional overlap bonus was added to reflect stronger evidence of interaction.

The numerical score was then mapped to three categories: low risk, medium risk, and high risk. A short warning message was also generated to make the result more interpretable. For example, if a cat were detected very close to a laptop, the system could output a high-risk warning.

During testing, weak detections sometimes caused false alarms. To reduce this, confidence thresholds were added so that very weak pet or object detections would be ignored before risk scoring.

Table 3. Risk logic and object weights.

Object class	Risk weight	Reason
Vase	55	Fragile and easy to break
Laptop	50	Expensive and easily damaged
Potted plant	45	Can be knocked over or spill soil
TV	40	Expensive household object that should be protected
Cup	35	Spill risk if a pet bumps into it
Remote	30	Common household object pets may interact with
Frisbee	20	Interaction is possible, but usually less harmful
Book	15	Lower risk compared with fragile or electronic objects

3. Results and Evaluation

a. Quantitative detector evaluation

The detector was evaluated on the held-out test set using standard object detection metrics, including $\text{mAP}@0.5$, $\text{mAP}@0.5:0.95$, precision, and recall. These metrics were appropriate because the project is fundamentally an object detection problem, so both localization quality and classification quality need to be measured.

The test results were:

- $\text{mAP}@0.5 = 0.3831$
- $\text{mAP}@0.5:0.95 = 0.2457$
- Precision = 0.4306
- Recall = 0.4213

These results show that the detector learned the selected classes to a moderate degree, but performance was not equally strong across all categories. Pet classes such as cat tended to

perform better than some of the smaller or more visually ambiguous object classes. This is likely due to class imbalance, object scale differences, and the difficulty of detecting cluttered indoor objects in varied scenes.

Table 4. Quantitative detector performance on the test set.

Metric	Value
mAP@0.5	0.3831
mAP@0.5:0.95	0.2457
Precision	0.4306
Recall	0.4213

b. Qualitative system evaluation

The full Pet Mischief Detector was also evaluated qualitatively by running the trained model on the held-out test images and saving annotated outputs. The output images also had bounding boxes along with a risk level and corresponding message, which resulted from the mischief logic.

As far as the quality of results goes, it can be concluded that if the initial detection was good, the warnings were reasonable at the scene level. If the system succeeded, the pets were spotted correctly, and so were other objects close to the household. For example, scenes in which a pet was detected close to a higher-risk object such as a laptop, TV, or potted plant produced believable high-risk warnings, while scenes with no detected pets or no nearby risky objects produced low-risk outputs.

The problems also arose at the stage of qualitative testing. The example of such a problem includes the detector’s inability to distinguish an area that looks like a wig from a dog, resulting in a false high mischief warning. One more example includes the situation where a dog was present in the background; however, the detector could not recognize it as a pet. Clearly, the effectiveness of the mischievousness detection depends heavily on the detection step being accurate.

While not without limitations, the qualitative outputs seem to have proven the validity of the logic for detecting mischief risk itself once detection was accurate. The weighting system, distance scores, and overlap bonus worked well together in creating intuitive alerts that reflected

the logic of mischief risks. However, the overall system was still less reliable in cluttered or visually ambiguous scenes, where false positives and missed detections were more likely.

Figure 5. Correct low-risk example.



Figure 6. Correct high-risk example

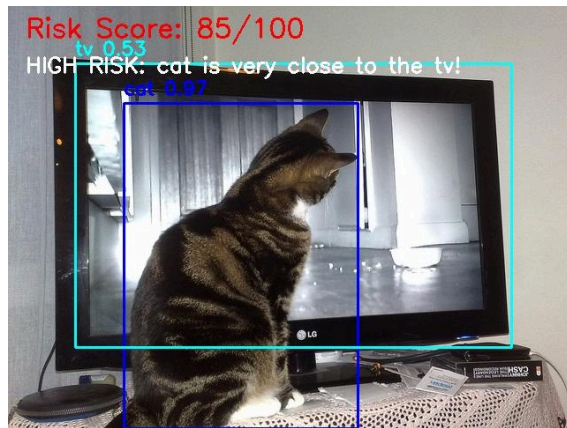


Figure 7: Failure case 1 – false positive



Figure 8: Failure case 2 – false negative



c. Robustness and failure analysis

To make the system more realistic, the curated dataset was built from diverse COCO scenes rather than overly clean or synthetic examples. As a result, the test set included cluttered backgrounds, multiple objects, varied viewpoints, and different scene layouts, making evaluation more challenging and closer to real-world use.

Even with all these opportunities for success, there were some failure instances experienced in certain situations. For example, some of the failures were due to false positive

and pet missing cases. The former occurred, for instance, where a wig-shaped object was detected and wrongly classified as a dog, hence resulting in high-risk alerts. Similarly, there were instances where a genuine dog appeared in the background, but it went undetected because the alert indicated the absence of any pets. From these examples, it is clear that the end outcome of the mischief alert is very dependent on the accuracy of the detector.

On the other hand, even though the detections may be faulty, the rule-based logic behind the mischief algorithm was quite accurate most times. False detections were filtered out by setting confidence thresholds to ensure stability of the outputs.

Overall, the system was more reliable in clearer scenes where pets and risky household objects were large and visible, and less reliable in cluttered scenes or scenes where objects and pets were partially occluded, small, or visually confusing.

4. Discussion

The project demonstrated that an end-to-end pet mischief detector can be built by combining object detection with simple spatial reasoning. One strength of the system is that it goes beyond object recognition alone and attempts to interpret context. The final risk score and warning message make the result more useful and human-readable than standard bounding box output.

On the other hand, the results of the conducted experiments proved that the whole system is dependent on the performance of the object detector. Since the determination of the risk directly relies on the output of the detector, namely classes and bounding boxes, even one mistake can change the overall perception of the situation in question.

The next limitation of our risk determination method is that its functioning is based solely on the use of geometric relationships between the detected objects. For example, two boxes may appear close even when the pet is not actually touching the object. Despite these limitations, the project successfully shows how computer vision methods can be extended with task-specific logic to solve a more context-aware problem. The system is not perfect, but it illustrates a clear pipeline from detection to interpretation.

5. Use of Generative AI

Generative AI was used as an implementation and debugging assistant during the project. It helped with generating starter code for dataset splitting, dataset export into YOLO format,

evaluation code, and the first version of the qualitative mischief logic. This was useful for speeding up repetitive programming tasks and for quickly building a working baseline pipeline.

However, the generated code was not always correct as-is. In practice, file paths, run names, and metric references sometimes had to be corrected manually so that the notebook matched the actual saved outputs. In addition, some early versions of the mischief logic accepted weak detections too easily, which produced obviously incorrect warnings. These issues were identified through manual testing and then fixed by adjusting paths, correcting code, and introducing confidence-based filtering.

This experience showed that GenAI can understand the overall structure of the project well and provide a strong starting point, but it still requires careful human verification. The final system depended on testing, debugging, and manual refinement rather than simply copying generated code directly.

6. Statement of Contribution

I was primarily responsible for Task 2 and Task 4 of the project. My main contribution was writing and refining the dataset splitting and formatting pipeline, including the reproducible train/validation/test split and the export of the dataset into YOLO format. I also implemented the quantitative evaluation code for the detector, designed and refined the mischief-logic module, generated the qualitative output images, and analyzed the system's failure cases.

Apart from these tasks, I also participated in debugging sections of the pipeline in the notebook as well as improving the final class set by contributing additional classes which could better describe the scenarios of pets getting into trouble. My partner took care of the training and tuning phase of the model itself.

7. Conclusion

In conclusion, this project developed a complete Pet Mischief Detector that combines object detection and simple spatial reasoning to assess scene-level risk. The system successfully detected pets and household objects and, in some cases, generated meaningful high-risk warnings. Quantitative evaluation showed moderate detector performance, while qualitative evaluation revealed both successful cases and important failure modes.

The most important lesson from the project is that the usefulness of the final mischief score depends strongly on the reliability of the detector. Even when the scene logic is sensible, false detections can still produce incorrect warnings. Future improvements could include

expanding and balancing the dataset, using a stronger detection model, or replacing bounding-box reasoning with more precise segmentation-based reasoning.

Overall, the project achieved its main goal of building an interpretable end-to-end system and provided a practical example of how computer vision can be combined with context-aware rules to solve a more meaningful real-world problem.